

Lecture 41-42: Solving Congruences

Zhilin Wu (吴志林),
Key Laboratory of System Software,
Institute of Software,
Chinese Academy of Sciences

Outline

- **Chinese Remainder Theorem**
- Back substitution
- Computer arithmetic with large integers
- Fermat's Little Theorem
- Primitive roots and discrete logarithms
- Applications of congruences
 - Hash functions
 - Pseudorandom numbers
 - Check digits

Chinese Remainder Theorem

How to solve systems of congruence equations ?

“今有物不知其数，三三数之剩二（除以3余2），
五五数之剩三（除以5余3），
七七数之剩二（除以7余2），问物几何？”（孙子算经）

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

Chinese Remainder Theorem

Chinese Remainder Theorem.

Let $m_1, \dots, m_n > 1$ be pairwise relatively prime positive integers and $a_1, \dots, a_n$ be arbitrary integers. Then

$$x \equiv a_1 \pmod{m_1},$$

$$x \equiv a_2 \pmod{m_2},$$

...

$$x \equiv a_n \pmod{m_n}$$

has a unique solution modulo $m = m_1 m_2 \dots m_n$.

Chinese Remainder Theorem

Chinese Remainder Theorem.

Let $m_1, \dots, m_n > 1$ be pairwise relatively prime positive integers and $a_1, \dots, a_n$ be arbitrary integers. Then

$$x \equiv a_1 \pmod{m_1},$$

$$x \equiv a_2 \pmod{m_2},$$

...

$$x \equiv a_n \pmod{m_n}$$

has a unique solution modulo $m = m_1 m_2 \dots m_n$.

There is a solution x with $0 \leq x < m$, and all other solutions x' satisfy $x' \equiv x \pmod{m}$.

Chinese Remainder Theorem

Lemma. Let $m_1, \dots, m_n > 1$ be pairwise relatively prime positive integers and $m = m_1 m_2 \dots m_n$.
For every $c \in \mathbb{Z}$, $m_i | c$ for every i implies $m | c$.

Proof.

Chinese Remainder Theorem

Proof of the theorem.

$$x \equiv a_1 \pmod{m_1},$$

$$x \equiv a_2 \pmod{m_2},$$

...

$$x \equiv a_n \pmod{m_n}$$

Chinese Remainder Theorem

Proof of the theorem (continued).

Chinese Remainder Theorem

Example. Solve

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 2 \pmod{7}.$$

Chinese Remainder Theorem

Example. Solve

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 2 \pmod{7}.$$

Outline

- Chinese Remainder Theorem
- **Back substitution**
- Computer arithmetic with large integers
- Fermat's Little Theorem
- Primitive roots and discrete logarithms
- Applications of congruences
 - Hash functions
 - Pseudorandom numbers
 - Check digits

Back Substitution

A **modular** way of solving congruence equations

$$x \equiv a_1 \pmod{m_1}, x \equiv a_2 \pmod{m_2}, \dots, x \equiv a_n \pmod{m_n}.$$

Example. Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$.

Back Substitution

procedure *backSubstitution* $((a_1, m_1), \dots, (a_n, m_n))$:

$m_1, \dots, m_n > 1$ are pairwise relatively prime,

$0 \leq a_1 < m_1, \dots, 0 \leq a_n < m_n$)

$c := m_1 \quad r := a_1$

for $i := 2$ **to** n

$\{x = cq_{i-1} + r \text{ and } c = m_1 \dots m_{i-1}\}$

$s := \bar{c}(a_i - r) \quad \{\bar{c} \text{ is an inverse of } c \text{ modulo } m_i\}$

$r := cs + r$

$c := cm_i$

return $r \quad \{c = m_1 \dots m_n = m \text{ and } x = cq_n + r, \text{ thus } x \equiv r \pmod{m}\}$

Back Substitution

procedure *backSubstitution* $((a_1, m_1), \dots, (a_n, m_n))$:

$m_1, \dots, m_n > 1$ are pairwise relatively prime,

$0 \leq a_1 < m_1, \dots, 0 \leq a_n < m_n$)

$c := m_1 \quad r := a_1$

for $i := 2$ **to** n

$\{x = cq_{i-1} + r \text{ and } c = m_1 \dots m_{i-1}\}$

$\{x = cq_{i-1} + r \equiv a_i \pmod{m_i} \Rightarrow q_{i-1} \equiv \bar{c}(a_i - r) \pmod{m_i}\}$

$s := \bar{c}(a_i - r) \quad \{\bar{c} \text{ is an inverse of } c \text{ modulo } m_i\}$

$\{q_{i-1} = m_i q_i + s, x = c(m_i q_i + s) + r = (cm_i)q_i + (cs + r)\}$

$r := cs + r$

$c := cm_i$

return $r \quad \{c = m_1 \dots m_n = m \text{ and } x = cq_n + r, \text{ thus } x \equiv r \pmod{m}\}$

Back Substitution

Example. Solve $x \equiv 2 \pmod{3}$, $x \equiv 1 \pmod{5}$, $x \equiv 3 \pmod{7}$.

Outline

- Chinese Remainder Theorem
- Back substitution
- **Computer arithmetic with large integers**
- Fermat's Little Theorem
- Primitive roots and discrete logarithms
- Applications of congruences
 - Hash functions
 - Pseudorandom numbers
 - Check digits

Computer arithmetic with large integers

Let $m_1, \dots, m_n$ be pairwise relatively prime positive integers
and $m = m_1 \dots m_n$.

Then every integer $0 \leq a < m$ can be uniquely represented by
 $(a \mathbf{mod} m_1, \dots, a \mathbf{mod} m_n)$.

$$(a_1, \dots, a_n) + (b_1, \dots, b_n) = (a_1 + b_1 \mathbf{mod} m_1, \dots, a_n + b_n \mathbf{mod} m_n),$$

$$(a_1, \dots, a_n) \cdot (b_1, \dots, b_n) = (a_1 \cdot b_1 \mathbf{mod} m_1, \dots, a_n \cdot b_n \mathbf{mod} m_n)$$

$$(a_1, \dots, a_n) \Rightarrow a: 0 \leq a < m$$

Solve $x \equiv a_1 \pmod{m_1}, \dots, x \equiv a_n \pmod{m_n}$.

Computer arithmetic with large integers

Let $m_1, \dots, m_n$ be pairwise relatively prime positive integers
and $m = m_1 \dots m_n$.

Then every integer $0 \leq a < m$ can be uniquely represented by
 $(a \mathbf{mod} m_1, \dots, a \mathbf{mod} m_n)$.

$$(a_1, \dots, a_n) + (b_1, \dots, b_n) = (a_1 + b_1 \mathbf{mod} m_1, \dots, a_n + b_n \mathbf{mod} m_n),$$

$$(a_1, \dots, a_n) \cdot (b_1, \dots, b_n) = (a_1 \cdot b_1 \mathbf{mod} m_1, \dots, a_n \cdot b_n \mathbf{mod} m_n)$$

$$(a_1, \dots, a_n) \Rightarrow a: 0 \leq a < m$$

Solve $x \equiv a_1 \pmod{m_1}, \dots, x \equiv a_n \pmod{m_n}$.

Faster computation than with the usual arithmetic, moreover,
computations for different components can be done **in parallel**.

Computer arithmetic with large integers

Example. Let $m_1 = 99$, $m_2 = 98$, $m_3 = 97$, and $m_4 = 95$. Then

$$m = 99 \cdot 98 \cdot 97 \cdot 95 = 89,403,930.$$

$$123,684 = (33, 8, 9, 89) \text{ and } 413,456 = (32, 92, 42, 16).$$

$$(33, 8, 9, 89) + (32, 92, 42, 16) =$$

$$(33 + 32 \text{ mod } 99, 8 + 92 \text{ mod } 98, 9 + 42 \text{ mod } 97, 89 + 16 \text{ mod } 95) \\ = (65, 2, 51, 10).$$

Solve

$$x \equiv 65 \pmod{99}, x \equiv 2 \pmod{98}, x \equiv 51 \pmod{97}, x \equiv 10 \pmod{95}.$$

Then we get $x \equiv 537,140 \pmod{m}$.

$(65, 2, 51, 10)$ represents the big integer 537,140.

Outline

- Chinese Remainder Theorem
- Back substitution
- Computer arithmetic with large integers
- **Fermat's Little Theorem**
- Primitive roots and discrete logarithms
- Applications of congruences
 - Hash functions
 - Pseudorandom numbers
 - Check digits

Fermat's Little Theorem

Fermat's Little Theorem.

If p is a prime, a is an integer **not** divisible by p , then

$$a^{p-1} \equiv 1 \pmod{p}.$$

Furthermore, for **every integer** a , $a^p \equiv a \pmod{p}$.

Remark. For every nonzero $a \in \mathbb{Z}_p = \{0, 1, \dots, p-1\}$,

$$a^{p-1} \equiv 1 \pmod{p}.$$

Fermat's Little Theorem

Proof.

Fermat's Little Theorem

Example. Compute $7^{222} \bmod 11$.

Outline

- Chinese Remainder Theorem
- Back substitution
- Computer arithmetic with large integers
- Fermat's Little Theorem
- **Primitive roots and discrete logarithms**
- Applications of congruences
 - Hash functions
 - Pseudorandom numbers
 - Check digits

Primitive Roots

A *primitive root modulo a prime p* is a nonzero integer $r \in \mathbb{Z}_p$ such that every nonzero element of $\mathbb{Z}_p$ is a power of r .

Primitive Roots

A *primitive root modulo a prime p* is a nonzero integer $r \in \mathbb{Z}_p$ such that every nonzero element of $\mathbb{Z}_p$ is a power of r .

Example. Determine whether 2 and 3 are the primitive roots of 11.

Primitive Roots

A *primitive root modulo a prime p* is a nonzero integer $r \in \mathbb{Z}_p$ such that every nonzero element of $\mathbb{Z}_p$ is a power of r .

Example. Determine whether 2 and 3 are the primitive roots of 11.

$$2^1 \bmod 11 = 2, 2^2 \bmod 11 = 4, 2^3 \bmod 11 = 8, 2^4 \bmod 11 = 5,$$

$$2^5 \bmod 11 = 10, 2^6 \bmod 11 = 9, 2^7 \bmod 11 = 7, 2^8 \bmod 11 = 3,$$

$$2^9 \bmod 11 = 6, 2^{10} \bmod 11 = 1$$

Primitive Roots

A *primitive root modulo a prime p* is a nonzero integer $r \in \mathbb{Z}_p$ such that every nonzero element of $\mathbb{Z}_p$ is a power of r .

Example. Determine whether 2 and 3 are the primitive roots of 11.

$$\begin{aligned} 2^1 \bmod 11 &= 2, 2^2 \bmod 11 = 4, 2^3 \bmod 11 = 8, 2^4 \bmod 11 = 5, \\ 2^5 \bmod 11 &= 10, 2^6 \bmod 11 = 9, 2^7 \bmod 11 = 7, 2^8 \bmod 11 = 3, \\ 2^9 \bmod 11 &= 6, 2^{10} \bmod 11 = 1 \end{aligned}$$

$$\begin{aligned} 3^1 \bmod 11 &= 3, 3^2 \bmod 11 = 9, 3^3 \bmod 11 = 5, 3^4 \bmod 11 = 4, \\ 3^5 \bmod 11 &= 1, 3^6 \bmod 11 = 3 \end{aligned}$$

Primitive Roots

A *primitive root modulo a prime p* is a nonzero integer $r \in \mathbb{Z}_p$ such that every nonzero element of $\mathbb{Z}_p$ is a power of r .

Example. Determine whether 2 and 3 are the primitive roots of 11.

$$\begin{aligned} 2^1 \bmod 11 &= 2, 2^2 \bmod 11 = 4, 2^3 \bmod 11 = 8, 2^4 \bmod 11 = 5, \\ 2^5 \bmod 11 &= 10, 2^6 \bmod 11 = 9, 2^7 \bmod 11 = 7, 2^8 \bmod 11 = 3, \\ 2^9 \bmod 11 &= 6, 2^{10} \bmod 11 = 1 \end{aligned}$$

$$\begin{aligned} 3^1 \bmod 11 &= 3, 3^2 \bmod 11 = 9, 3^3 \bmod 11 = 5, 3^4 \bmod 11 = 4, \\ 3^5 \bmod 11 &= 1, 3^6 \bmod 11 = 3 \end{aligned}$$

2 is a primitive root of 11, while 3 is not.

Primitive Roots

Proposition. If r is a primitive root modulo a prime p , then $r^1 \bmod p, \dots$, and $r^{p-1} \bmod p$ are mutually distinct.

Proof. Because $r \in \mathbb{Z}_p = \{0, 1, \dots, p-1\}$ is nonzero,

from Fermat's Little Theorem, $r^{p-1} \equiv 1 \pmod{p}$.

Then $r^1 \bmod p, \dots, r^{p-1} \bmod p$ exhaust all powers of r modulo p .

For contradiction, suppose $r^i \bmod p = r^j \bmod p$ for some $1 \leq i < j \leq p-1$.

Then $r^1 \bmod p, \dots, r^{j-1} \bmod p$ with $j-1 < p-1$ exhaust all powers of r modulo p , contradicting the assumption that

all $p-1$ nonzero elements of $\mathbb{Z}_p$ are powers of r modulo p .

Primitive Roots

Proposition. If r is a primitive root modulo a prime p , then $r^1 \bmod p, \dots$, and $r^{p-1} \bmod p$ are mutually distinct.

Proof. Because $r \in \mathbb{Z}_p = \{0, 1, \dots, p-1\}$ is nonzero,

from Fermat's Little Theorem, $r^{p-1} \equiv 1 \pmod{p}$.

Then $r^1 \bmod p, \dots, r^{p-1} \bmod p$ exhaust all powers of r modulo p .

For contradiction, suppose $r^i \bmod p = r^j \bmod p$ for some $1 \leq i < j \leq p-1$.

Then $r^1 \bmod p, \dots, r^{j-1} \bmod p$ with $j-1 < p-1$ exhaust all powers of r modulo p , contradicting the assumption that

all $p-1$ nonzero elements of $\mathbb{Z}_p$ are powers of r modulo p .

Theorem. There is a primitive root r modulo p for every prime p .

Discrete Logarithm

Suppose that p is a prime, r is a primitive root modulo p , and

$$1 \leq a \leq p-1.$$

If $r^e \bmod p = a$ with $1 \leq e \leq p-1$, then

e is the *discrete logarithm* of a modulo p to the base r ,

denoted by $e = \log_r a \pmod{p}$.

$\log_r a \pmod{p}$ can be simplified into $\log_r a$ if p is clear from the context.

Discrete Logarithm

Suppose that p is a prime, r is a primitive root modulo p , and

$$1 \leq a \leq p-1.$$

If $r^e \bmod p = a$ with $1 \leq e \leq p-1$, then

e is the *discrete logarithm* of a modulo p to the base r ,

denoted by $e = \log_r a \pmod{p}$.

$\log_r a \pmod{p}$ can be simplified into $\log_r a$ if p is clear from the context.

Example.

Find the discrete logarithm of 3 and 5 modulo 11 to the base 2.

$$2^8 \bmod 11 = 3, 2^4 \bmod 11 = 5.$$

Thus $8 = \log_2 3 \pmod{11}$ and $4 = \log_2 5 \pmod{11}$.

Outline

- Chinese Remainder Theorem
- Back substitution
- Computer arithmetic with large integers
- Fermat's Little Theorem
- Primitive roots and discrete logarithms
- **Applications of congruences**
 - Hash functions
 - Pseudorandom numbers
 - Check digits

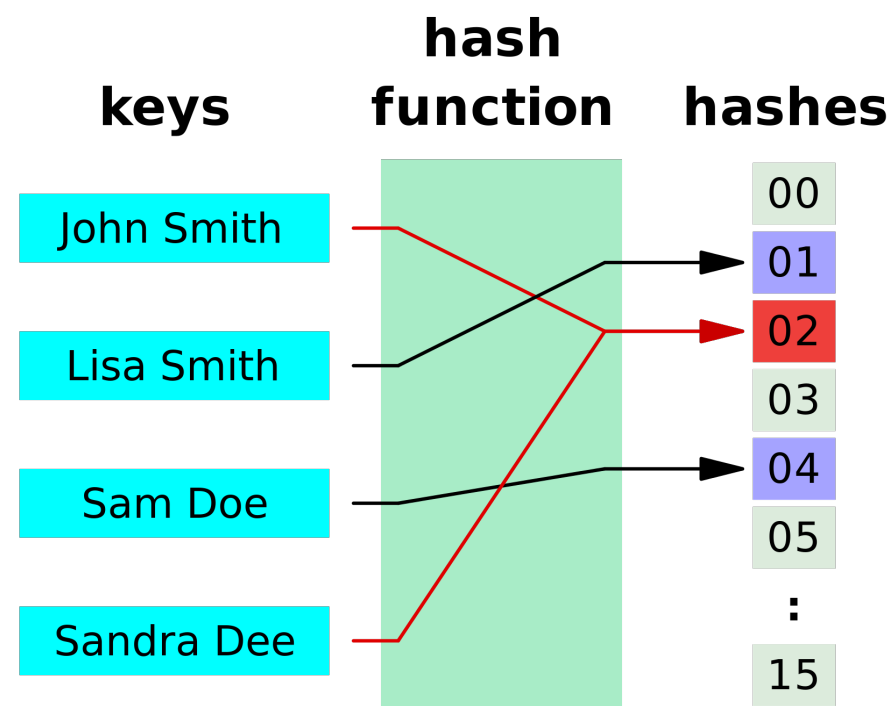
Hash Functions



Hash Functions

Hash function

Any function that can be used to map data of **arbitrary size** to **fixed-size** values



Used in data storage and retrieval applications to **access data in a small and nearly constant time per retrieval.**

Hash Functions

$$h(k) = k \bmod m$$

For instance,

if keys are 身份证号 with 18 decimal digits and $m = 1021$,
then $h(430820198305041872) = 592$.

When a *collision* occurs, e.g. $h(430820198305031662) = 592$,
then one way of resolving the collision is to
use the **first free position** after 592, e.g. 593.

Pseudorandom Numbers

Randomly chosen numbers:

Often needed for computer simulations

Sequences that are closer to truly random:

Can be generated using hardware random number generators

Pseudorandom Numbers

Randomly chosen numbers:

Often needed for computer simulations

Sequences that are **closer to truly random**:

Can be generated using **hardware** random number generators

Pseudorandom numbers:

A sequence of numbers whose properties **approximate**
the properties of sequences of random numbers.

Pseudorandom Numbers

Randomly chosen numbers:

Often needed for computer simulations

Sequences that are closer to truly random:

Can be generated using hardware random number generators

Pseudorandom numbers:

A sequence of numbers whose properties approximate the properties of sequences of random numbers.

Pseudorandom number generators:

Important in practice for their speed in number generation and their reproducibility.

Linear congruential method: 线性同余法

modulus: 模数, multiplier: 乘数, increment: 增量, seed: 种子

Pseudorandom Numbers

Linear congruential method:

Most commonly used procedure
for generating pseudorandom numbers

Linear congruential method: 线性同余法

modulus: 模数, multiplier: 乘数, increment: 增量, seed: 种子

Pseudorandom Numbers

Linear congruential method:

Most commonly used procedure
for generating pseudorandom numbers

Four integers:

modulus m , multiplier a , increment c , seed x_0 ,
with $2 \leq a < m$, $0 \leq c < m$, and $0 \leq x_0 < m$.

Linear congruential method: 线性同余法

modulus: 模数, multiplier: 乘数, increment: 增量, seed: 种子

Pseudorandom Numbers

Linear congruential method:

Most commonly used procedure
for generating pseudorandom numbers

Four integers:

modulus m , multiplier a , increment c , seed x_0 ,
with $2 \leq a < m$, $0 \leq c < m$, and $0 \leq x_0 < m$.

Generate a sequence of numbers $\{x_n\}$ by using

$$x_{n+1} = (ax_n + c) \bmod m.$$

Pseudorandom Numbers

Linear congruential method

Pseudorandom Numbers

Linear congruential method

Example. Let $m = 9$, $a = 7$, $c = 4$, and $x_0 = 3$. Then

$$x_1 = (7x_0 + 4) \bmod 9 = 7,$$

$$x_2 = (7x_1 + 4) \bmod 9 = 8,$$

$$x_3 = (7x_2 + 4) \bmod 9 = 6,$$

$$x_4 = (7x_3 + 4) \bmod 9 = 1,$$

$$x_5 = (7x_4 + 4) \bmod 9 = 2,$$

$$x_6 = (7x_5 + 4) \bmod 9 = 0,$$

$$x_7 = (7x_6 + 4) \bmod 9 = 4,$$

$$x_8 = (7x_7 + 4) \bmod 9 = 5,$$

$$x_9 = (7x_8 + 4) \bmod 9 = 3.$$

Pseudorandom Numbers

Linear congruential method

Example. Let $m = 9$, $a = 7$, $c = 4$, and $x_0 = 3$. Then

$$x_1 = (7x_0 + 4) \bmod 9 = 7,$$

$$x_2 = (7x_1 + 4) \bmod 9 = 8,$$

$$x_3 = (7x_2 + 4) \bmod 9 = 6,$$

$$x_4 = (7x_3 + 4) \bmod 9 = 1,$$

$$x_5 = (7x_4 + 4) \bmod 9 = 2,$$

$$x_6 = (7x_5 + 4) \bmod 9 = 0,$$

$$x_7 = (7x_6 + 4) \bmod 9 = 4,$$

$$x_8 = (7x_7 + 4) \bmod 9 = 5,$$

$$x_9 = (7x_8 + 4) \bmod 9 = 3.$$

3, 7, 8, 6, 1, 2, 0, 4, 5, 3, 7, 8, 6, 1, 2, 0, 4, 5, 3, ...

Pseudorandom Numbers

Pure multiplier generator

Linear congruential method with $c = 0$.

Pseudorandom Numbers

Pure multiplier generator

Linear congruential method with $c = 0$.

For instance,

pure multiplier generator with

$m = 2^{31}-1$ and $a = 7^5 = 16,807$

are widely used, and

$2^{31}-2$ numbers are generated before repetition begins.

Check Digits

Check errors in digit strings:

Add a **check digit** at the end.

Check a digit string is correct:

Check whether this final digit has the correct value

Check Digits

International Standard Book Number (ISBN)

13 digits, with the last digit a check digit



Check Digits



Compute the check digit of ISBN

$$(x_1 + 3x_2 + x_3 + 3x_4 + x_5 + 3x_6 + x_7 + 3x_8 + x_9 + 3x_{10} + x_{11} + 3x_{12} + x_{13}) \\ \equiv 0 \pmod{10}$$

$$x_{13} \equiv$$

$$(-x_1 - 3x_2 - x_3 - 3x_4 - x_5 - 3x_6 - x_7 - 3x_8 - x_9 - 3x_{10} - x_{11} - 3x_{12}) \pmod{10}$$

$$x_{13} \equiv$$

$$(-9 - 3 \cdot 7 - 8 - 3 \cdot 7 - 1 - 3 \cdot 1 - 1 - 3 \cdot 3 - 8 - 3 \cdot 5 - 5 - 3 \cdot 0)$$

$$\equiv -101 \equiv 9 \pmod{10}$$

Check Digits

Detect *single error*: An error in **one** digit

Let $x_1, \dots, x_{13}$ be a correct ISBN, and

the j -th digit be changed to $(x_j + a) \bmod 10$ with $1 \leq a \leq 9$.

If j is odd, then

$$-\left(\sum_{i \equiv 1 \pmod{2}} x_i + \sum_{i' \equiv 0 \pmod{2}} 3x_{i'} + a \right) = (x_{13} - a) \bmod 10 \neq x_{13} \bmod 10.$$

Otherwise,
$$-\left(\sum_{i \equiv 1 \pmod{2}} x_i + \sum_{i' \equiv 0 \pmod{2}} 3x_{i'} + 3a \right) \bmod 10$$

$$= (x_{13} - 3a) \bmod 10 \neq x_{13} \bmod 10,$$

since $3 \leq 3a \leq 27$, $3a \neq 20, 10$, thus $3a \not\equiv 0 \pmod{10}$.

Check Digits

Detect *transposition error*:

Two adjacent digits are **interchanged**.

Let $x_1, \dots, x_{13}$ be a correct ISBN.

Suppose that x_j and x_{j+1} are interchanged.

If j is odd, then $x_j + 3x_{j+1}$ is changed to $x_{j+1} + 3x_j$,

and $x_{j+1} + 3x_j - x_j - 3x_{j+1} = 2(x_j - x_{j+1})$.

Otherwise, $3x_j + x_{j+1}$ is changed to $3x_{j+1} + x_j$,

and $3x_{j+1} + x_j - 3x_j - x_{j+1} = 2(x_{j+1} - x_j)$.

Check Digits

Detect *transposition error*:

Two adjacent digits are **interchanged**.

Let $x_1, \dots, x_{13}$ be a correct ISBN.

Suppose that x_j and x_{j+1} are interchanged.

If j is odd, then $x_j + 3x_{j+1}$ is changed to $x_{j+1} + 3x_j$,

and $x_{j+1} + 3x_j - x_j - 3x_{j+1} = 2(x_j - x_{j+1})$.

Otherwise, $3x_j + x_{j+1}$ is changed to $3x_{j+1} + x_j$,

and $3x_{j+1} + x_j - 3x_j - x_{j+1} = 2(x_{j+1} - x_j)$.

If $x_j - x_{j+1} = 5$ or -5 , then $2(x_j - x_{j+1}) = 10$ or -10 and the transposition error will **not** be discovered.

Recap

Chinese Remainder Theorem.

Let $m_1, \dots, m_n > 1$ be pairwise relatively prime positive integers and $a_1, \dots, a_n$ be arbitrary integers. Then

$$x \equiv a_1 \pmod{m_1},$$

$$x \equiv a_2 \pmod{m_2},$$

...

$$x \equiv a_n \pmod{m_n}$$

has a unique solution modulo $m = m_1 m_2 \dots m_n$.

procedure *backSubstitution*((a_1, m_1), ..., (a_n, m_n):

$m_1, \dots, m_n > 1$ are pairwise relatively prime)

$c := m_1$ $r := a_1$

for $i := 2$ **to** n

$\{x = cq_{i-1} + r\}$

$s := \bar{c}(a_i - r)$ $\{\bar{c}$ is an inverse of c modulo $m_i\}$

$\{q_{i-1} = m_i q_i + s, x = c(m_i q_i + s) + r\}$

$r := cs + r$

$c := cm_i$

return r $\{c = m_1 \dots m_n = m$ and $x = cq_n + r$, thus $x \equiv r \pmod{m}\}$

Fermat's Little Theorem.

If p is a prime, a is an integer **not** divisible by p , then

$$a^{p-1} \equiv 1 \pmod{p}.$$

Furthermore, for **every integer** a , $a^p \equiv a \pmod{p}$.

Recap

$(a \bmod m_1, \dots, a \bmod m_n)$

$$(a_1, \dots, a_n) + (b_1, \dots, b_n) = (a_1 + b_1 \bmod m_1, \dots, a_n + b_n \bmod m_n)$$

- Computer arithmetic with large integers

- Fermat's Little Theorem

Compute $a^n \bmod p$ for prime p and a such that $p \nmid a$

$$a^n \equiv a^{n \bmod p-1} \pmod{p}.$$

- Primitive roots and discrete logarithms

Theorem. There is a primitive root r modulo p for every prime p .

- Applications of congruences

- Hash functions

$$h(k) = k \bmod m$$

- Pseudorandom numbers

$$x_{n+1} = (ax_n + c) \bmod m$$

- Check digits

A primitive root modulo a prime p is an integer $r \in \mathbb{Z}_p$ such that every nonzero elements of $\mathbb{Z}_p$ is a power of r .

Suppose that p is a prime, r is a primitive root modulo p , and

$$1 \leq a \leq p-1.$$

If $r^e \bmod p = a$ and $0 \leq e \leq p-1$, then

e is the *discrete logarithm* of a modulo p to the base r ,

denoted by $\log_r a \pmod{p} = e$.

$$\text{ISBN } (x_1 + 3x_2 + x_3 + 3x_4 + x_5 + 3x_6 + x_7 + 3x_8 + x_9 + 3x_{10} + x_{11} + 3x_{12} + x_{13}) \equiv 0 \pmod{10}$$

Homework

1. Section 4.4.

- Page 301, Exercise 20.
- Page 302, Exercise 28.
- Page 302, Exercise 38.

2. Section 4.5.

- Page 310, Exercise 34.

作业和答疑

1. 交作业时间：奇数周周一上课前（两周交一次）
2. 交作业方式（二选一）：
 - ◆ 通过<https://mooc.ucas.edu.cn/portal>课程网站
 - ◆ 上课现场提交
3. 答疑：助教崔乐天、刘易铖

Next Lecture

Solving Linear Homogeneous
Recurrence Relations